

Management Tools

Managing the implementation process

Version management (Git)

- Coordinate development progress
- Code changes become easily reversible

Build system (Maven/Gradle)

- Automate the process of assembling the system from its sources

Issue tracking (GitHub)

- Keep track of bug reports and requests for new system feature

Git for Version Control

Created by Linus Torvalds,
creator of Linux, in 2005

- Came out of Linux development community
- Designed to do version control

Goals of Git:

- Speed
- Support for non-linear development

(thousands of parallel branches)

- Fully distributed
- Able to handle large projects efficiently



Installing/learning Git

Git website:

- Free on-line book: <http://git-scm.com/book>

Git for Computer Scientists:

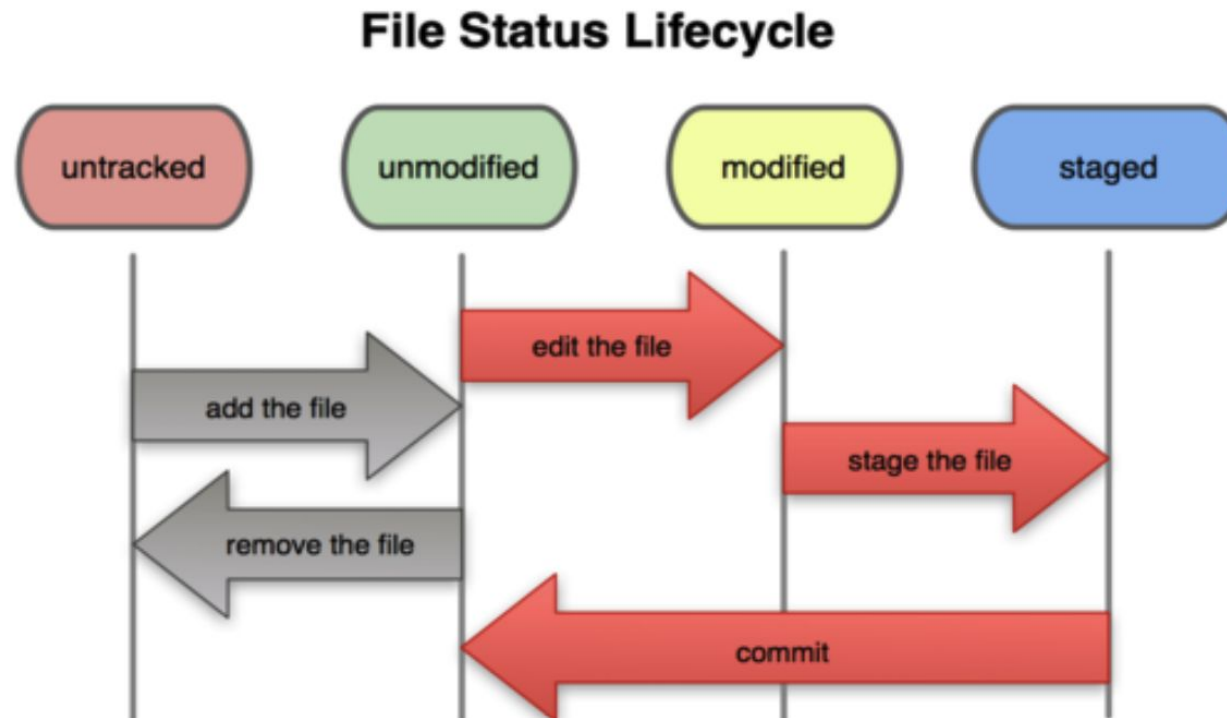
- <http://eagain.net/articles/git-for-computer-scientists/>
- At command line: (where verb = config, add, commit, etc.)
- git help verb

Basic Git workflow

Modify files in your working directory.

Stage files, adding snapshots of them to your staging area.

Commit, which takes the files in the staging area and stores that snapshot permanently to your Git directory.



Initial Git configuration

Set the name and email for Git to use when you commit:

- `git config --global user.name "Bugs Bunny"`
- `git config --global user.email bugs@gmail.com`
- You can call `git config --list` to verify these are set.

Creating a Git repo

To create a new local Git repo in your current directory:

- `git init`

This will create a `.git` directory in your current directory.

Then you can commit files in that directory into the repo.

- `git add filename`
- `git commit -m "commit message"`

To clone a remote repo to your current directory:

- `git clone url localDirectoryName`

This will create the given local directory, containing a working copy of the files from the repo, and a `.git` directory (used to hold the staging area and your actual local repo)

Git commands

command	description
<code>git clone <i>url</i> [<i>dir</i>]</code>	copy a Git repository so you can add to it
<code>git add <i>file</i></code>	adds file contents to the staging area
<code>git commit</code>	records a snapshot of the staging area
<code>git status</code>	view the status of your files in the working directory and staging area
<code>git diff</code>	shows diff of what is staged and what is modified but unstaged
<code>git help [<i>command</i>]</code>	get help info about a particular command
<code>git pull</code>	fetch from a remote repo and try to merge into the current branch
<code>git push</code>	push your new branches and data to a remote repository
others: <code>init</code> , <code>reset</code> , <code>branch</code> , <code>checkout</code> , <code>merge</code> , <code>log</code> , <code>tag</code>	

Managing the implementation process

Version management (Git)

- Coordinate development progress
- Code changes become easily reversible

Build system (Maven/Gradle)

- Automate the process of assembling the system from its sources

Issue tracking (GitHub)

- Keep track of bug reports and requests for new system feature

Maven

<https://maven.apache.org/>



- Apache Maven is a software project management and comprehension tool.
- Based on the concept of a **Project Object Model (POM)**, Maven can manage a project's build, reporting and documentation from a central piece of information.
- Maven is a project management and build automation tool mainly used for **Java** projects.

Project Object Model (POM)

- An XML file that contains information about the project and configuration details
- When executing a task or goal, Maven looks for the POM in the current directory. It reads the POM, gets the needed configuration information, then executes the goal.

Project Object Model (POM)

- The minimum requirement for a POM are the following:
 - project root
 - `modelVersion` - should be set to 4.0.0
 - `groupId` - the id of the project's group.
 - `artifactId` - the id of the artifact (project)
 - `version` - the version of the artifact under the specified group

1. `<project>`

2. `<modelVersion>4.0.0</modelVersion>`

3.

4. `<groupId>com.mycompany.app</groupId>`

5. `<artifactId>my-app</artifactId>`

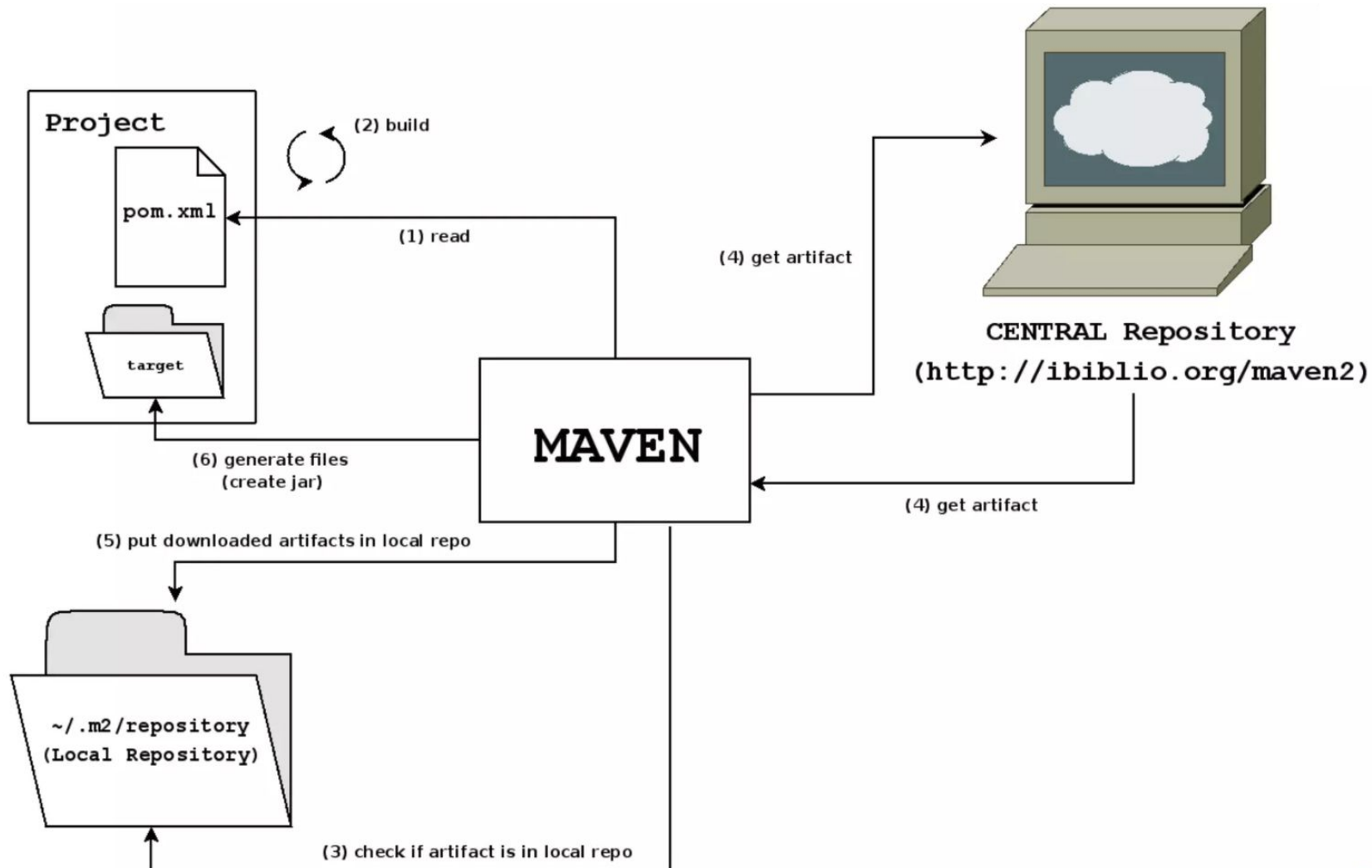
6. `<version>1</version>`

7. `</project>`

What Maven can do?

- Model our software project
- Centralize project information
- Manage our build process
- Gather data about our software project and the build itself
- Document the software, and our project

How Maven works?



Maven Command

mvn --version	Prints out the version of Maven you are running.
mvn clean	Clears the target directory into which Maven normally builds your project.
mvn package	Builds the project and packages the resulting JAR file into the target directory.
mvn clean package	Clears the target directory and Builds the project and packages the resulting JAR file into the target directory.
mvn clean package -Dmaven.test.skip=true	Clears the target directory and builds the project and packages the resulting JAR file into the target directory - without running the unit tests during the build.
mvn verify	Runs all integration tests found in the project.
mvn clean verify	Cleans the target directory, and runs all integration tests found in the project.
mvn install	Builds the project described by your Maven POM file and installs the resulting artifact (JAR) into your local Maven repository
mvn clean install	Clears the target directory and builds the project described by your Maven POM file and installs the resulting artifact (JAR) into your local Maven repository

-Dmaven.test.skip=true

Skip tests

What is Gradle?

- Gradle is an open source build automation system that builds upon the concepts and Apache Ant and Apache Maven and introduces a **domain-specific language (DSL)** instead of the XML form used by Apache Maven for declaring the project configuration.
- It is widely used in Java, Android, and multi-language projects.



Domain specific language (Groovy)

Groovy lets you omit parentheses around the arguments of a method call for top-level statements. "**command chain**" feature extends this by allowing us to chain such parentheses-free method calls, requiring neither parentheses around arguments, nor dots between the chained calls.

`a b c d` will actually be equivalent to `a(b).c(d)`.

DSL or Domain specific language is meant to simplify the code written in Groovy in such a way that it becomes easily understandable for the common user.

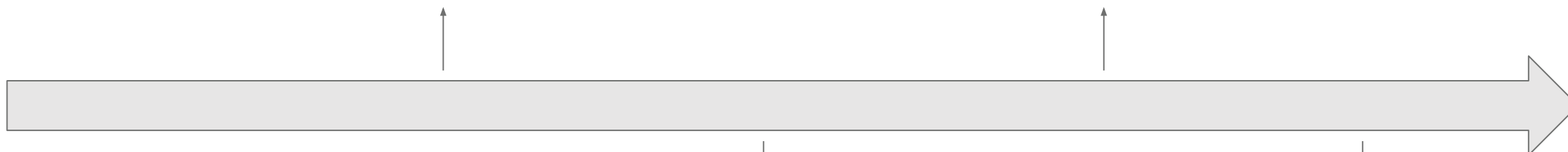
Brief History of Gradle

Birth of Gradle (2007–2009):

- Use a Groovy-based DSL instead of XML
- Gradle 1.0 officially released in 2012.

Introduction of Kotlin DSL

(2016–2018): Kotlin DSL
(`build.gradle.kts`)



Before Gradle: Ant (2000) and
Maven Era (2004)

Groovy DSL Era (2012–2016):
`build.gradle` written in
Groovy

Modern Gradle (2019 – Present):
Gradle is now standard for Android
development and large enterprise
systems

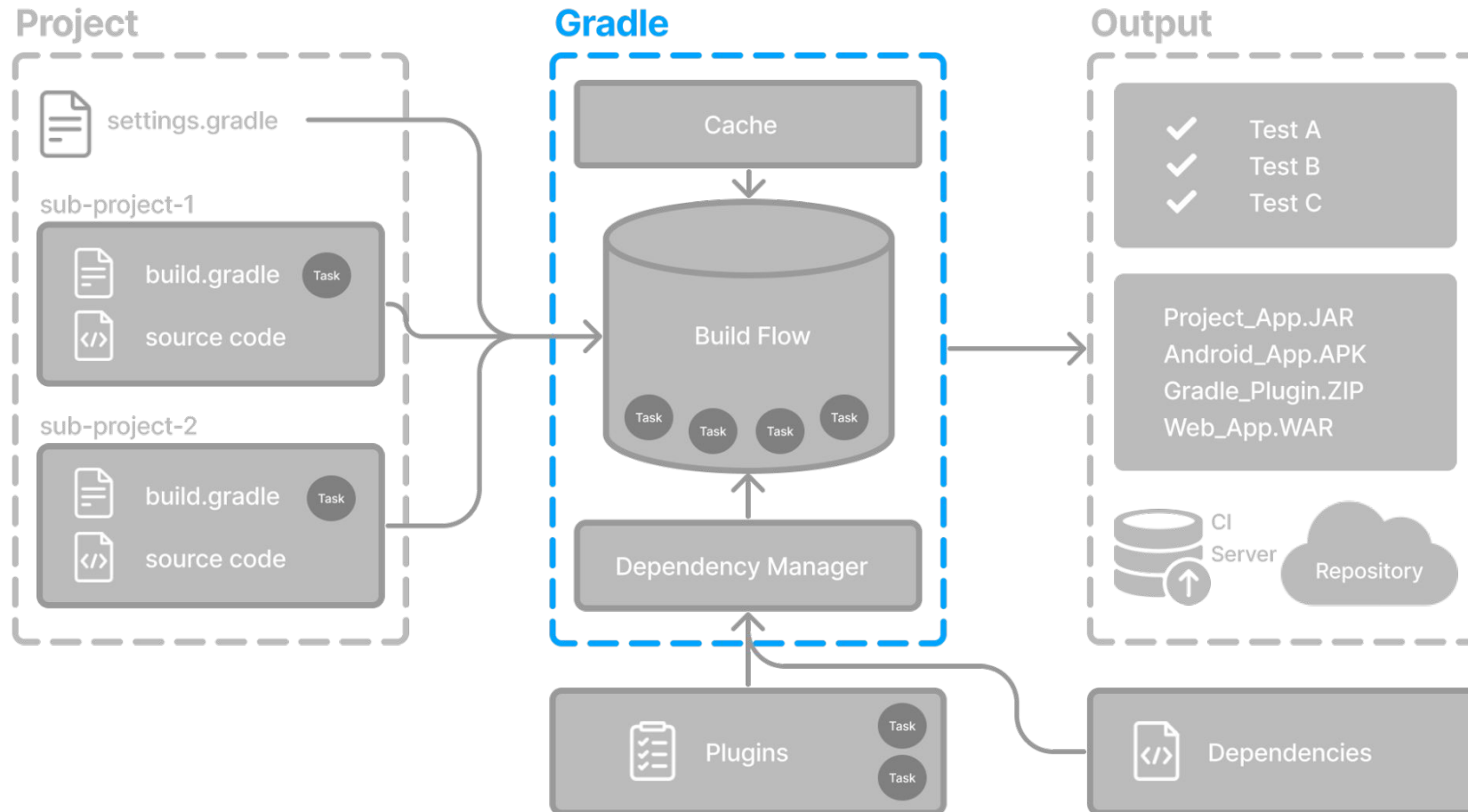
Why Gradle?

- **Polyglot builds:** LinkedIn uses Gradle to build 60 different programming languages including Java, Scala, Python, C/C++, Android, IOS and many more.
- **Robust dependency management:** out of the box, Gradle handles transitive dependencies that resolve across multiple repository types including Maven, Ivy, flat files.
- **Tool integrations:** Plugins and integrations with every imaginable tool in the DevOps Automation pipeline from IDEs like Eclipse, Android Studio and IntelliJ to Jenkins through to Chef, Puppet, Docker, and Ansible.

Why Gradle?

Aspect	Maven	Gradle
Configuration language	XML	Groovy or Kotlin
Build model	Fixed and linear phases	Flexible and dynamic task graph
Dependency management	Strong	Strong
Plugin system	Large	Large
Convention over configuration	Yes	Yes
Customization	Limited	High
Performance	Poor	Good
Scalability	Low	High
Incrementality	No	Yes
Caching	No	Yes
Parallelism	Yes	Yes
Language support	Java only	Multiple languages
Platform support	Java only	Multiple platforms
IDE integration	Good	Fair
Documentation	Good	Good
Community	Large	Large

How Gradle works?



Gradle Command

<code>gradle init</code>	Convert Maven build to gradle
<code>gradle -q projects</code>	List all projects in the build
<code>./gradlew clean build --parallel</code>	Execute tasks parallel
<code>gradle build -x test</code>	Build project and skip test

Gradle Wrapper:

The recommended way to execute any Gradle build is with the help of the Gradle Wrapper (referred to as "Wrapper").

The Wrapper is a script that invokes a declared version of Gradle, downloading it beforehand if necessary. As a result, developers can get up and running with a Gradle project quickly.

Gradle Wrapper

- Instead of running gradle build using the installed Gradle, you use the Gradle Wrapper by calling `./gradlew build`.
- Gradle Wrapper provides a consistent build environment by automatically managing the Gradle version, reducing setup effort and improving build reliability.

Managing the implementation process

Version management (Git)

- Coordinate development progress
- Code changes become easily reversible

Build system (Maven/Gradle)

- Automate the process of assembling the system from its sources

Issue tracking (GitHub)

- Keep track of bug reports and requests for new system feature

GitHub

Tutorial:

<https://docs.github.com/en/get-started/start-your-journey/hello-world>

Example:

<https://github.com/apache/dubbo>

Thank you !